Real-Trace Offline Evaluation of Budget-Constrained Selective Runtime Monitoring

CS 6158 Project: Shiyun Cheng and Gengkun Shao

October 18, 2026

Abstract

We evaluate five budgeted monitor-selection strategies using historical TraceMOP-derived data from two Java projects. Candidate groups, recorded trace identifiers, frequencies, and event-check cost proxies originate from an experimental workbook. At a 20% proxy budget, utility-per-cost and redundancy-aware greedy both cover 97.8% of observed unique trace IDs for `javadbf` and 99.7% for `jlemmagen`. These are *offline observed-trace coverage* measurements, not deployed monitoring recall, violation recall, or measured runtime savings.

1 Research question and scope

Our original project asks whether historical information about monitoring cost, usefulness, and redundancy can allocate a limited monitoring budget more effectively than simple baselines or Valg, particularly at comparable *realized* overhead. This experiment addresses one intermediate question: how much of the previously observed distinct trace information could candidate subsets cover under a budget based on an event-check cost proxy? It does not yet test independently disabled/enabled creation-site monitors or compare against Valg's adaptive online policy.

2 Dataset and preprocessing

We use the `Monitor profile` worksheet of `valg-algorithm-input.xlsx`. Each worksheet row is treated as one candidate *location* × *monitoring specification group*. The purported creation location is inferred from the first trace event, so it is not verified as an actual instrumentable creation site. Each candidate i has a recorded frequency f_i, a positive cost proxy c_i (`cost_proxy_event_checks`), and a set T_i of distinct recorded trace identifiers parsed from `trace_ids`. Trace identifiers are unioned within each project; duplication across groups is counted once in coverage.

Project	Candidate groups	Observed distinct trace IDs	Total cost proxy
`javadbf`	153	278	713,840
`jlemmagen`	66	2,115	8,572,328

For project p, let $U_p = \bigcup_i T_i$ and $C_p = \sum_i c_i$. For each budget fraction $\beta \in \{0.10, 0.20, 0.30, 0.50, 0.80\}$, we select S satisfying $\sum_{i \in S} c_i \le \beta C_p$. Our outcome measure is

$$\text{ObservedTraceRecall}(S) = \frac{\left|\bigcup_{i \in S} T_i\right|}{|U_p|}.$$

This denominator contains only trace IDs already present in the workbook: it is not the universe of all feasible execution traces. The proxy budget is in event-check units, *not milliseconds, memory bytes, or realized total monitoring overhead.*

3 Algorithms and reproducible protocol

We compare (1) **Random**, shuffle candidates and include each item that fits; (2) **Frequency**, rank by f_i; (3) **Utility/cost**, rank by $|T_i|/c_i$; (4) **Redundancy-aware greedy**, repeatedly choose the affordable candidate maximizing $|T_i \setminus \bigcup_{j \in S} T_j|/c_i$, stopping when no positive marginal coverage fits; and (5) **Additive knapsack**, run discrete 0–1 knapsack maximizing $\sum_{i \in S} |T_i|$, with costs rounded upward for budget feasibility. Knapsack's internal objective *double-counts overlapping trace IDs*; final recall is always computed by the union.

Each deterministic method is run once for each project–budget pair. Random is repeated 30 times per pair, using seeds 42–71; reported random results are means and sample standard deviations of *selection variability on the same data*, not cross-program or timing uncertainty. All methods recompute a selection independently for each budget, rather than forcing larger-budget selections to contain smaller-budget selections. The executable implementation is `run_experiments.py`; outputs include per-trial metrics, aggregate metrics, selections, and plots.

4 Results

Table 1: Observed unique-trace recall (%) by budget. Random entries are mean ± SD over 30 trials.

Project	Budget	Random	Frequency	Utility/cost	Redundancy greedy	Additive knapsack
`javadbf`	10%	89.6±0.0	89.6	89.6	89.6	70.5
	20%	86.5±14.2	45.0	97.8	97.8	89.6
	30%	90.7±2.6	11.2	97.8	97.8	96.8
	50%	85.2±15.5	89.2	98.9	98.9	98.9
	80%	99.1±1.6	99.6	99.6	99.6	99.3
`jlemmagen`	10%	43.1±38.9	88.4	99.5	99.5	99.3
	20%	65.6±38.0	88.8	99.7	99.7	99.6
	30%	64.0±39.0	1.8	99.8	99.8	99.7
	50%	82.4±32.3	70.9	99.9	99.9	99.8
	80%	90.2±24.5	99.8	100.0	100.0	99.9

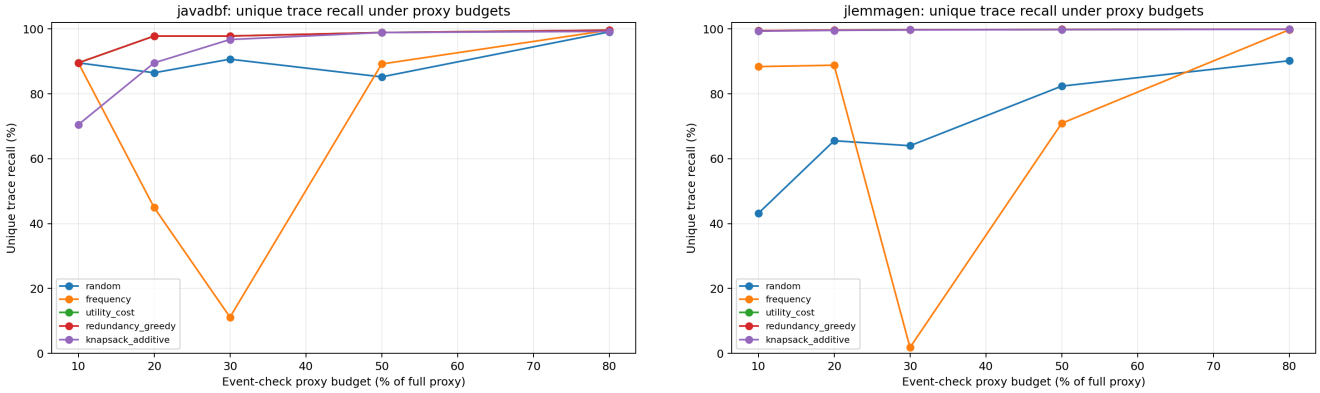


Figure 1: Coverage of historical trace IDs vs. percent of full event-check proxy budget. Curves are not empirical runtime–recall tradeoffs.

Interpretation. At 20%, both utility/cost and redundancy-aware greedy obtain 97.8% for `javadbf` and 99.7% for `jlemmagen`. Their near-identical performance here provides no evidence that overlap-aware greedy is superior to static utility/cost on these workloads. Additive knapsack can perform worse because it optimizes a sum of standalone trace counts and rewards duplicate coverage multiple times. For example, its `javadbf` recall is 89.6% at 20%, below both coverage-aware methods.

The *non-monotonic* frequency curve is not necessarily a data or coding error: at every new budget, a fixed ordered list is rescanned with a fit-or-skip rule. A larger budget may now admit a costly group early in that list, leaving

insufficient budget for many inexpensive groups that were previously selected; hence chosen sets are not nested. This is particularly visible at `jlemmagen`'s 30% budget (1.8% recall). The large random standard deviations on `jlemmagen` likewise suggest that coverage is concentrated in particular candidate groups. On `javadbf` at 10%, almost all 153 groups fit and the random trials happen to produce identical recall; this reflects the highly skewed proxy-cost distribution, not robust performance across datasets.

5 Limitations and next steps

These results are descriptive, in-sample, and offline. There is no separation of historical profiling executions from held-out evaluation executions. The creation-site assumption is unverified, and aggregated group trace sets do not establish what would remain observable if those creation sites were independently disabled. No per-group violation-signature attribution is available, so we do not measure violation recall or violation-weighted utility. Proxy event-check totals cannot support claims about actual elapsed time, memory use, or matched-overhead superiority to Valg. Cost additivity is also unvalidated.

Next we will (i) validate candidate groups against actual monitor-creation locations; (ii) add per-site enable/disable controls and test trace/violation preservation on held-out runs; (iii) measure runtime and memory for complete selected configurations; (iv) compare all methods against Valg at *matched realized cost*; and, if possible, (v) test profiles trained on one revision against later revisions and add an exploration budget. If independent control is infeasible, we will clearly label a fallback based on profiling proxies as an offline simulation rather than a full runtime evaluation.